Proposal: Verified Peephole Optimizer for Arithmetic Expressions in Lean 4

HONGXU XU*, Cheriton School of Computer Science, University of Waterloo, Canada

Formal methods have been increasingly applied to verify the correctness of compiler optimizations, ensuring that transformations preserve program semantics. This proposal outlines a project to formalize LLVM peephole optimizations in Lean4, a powerful proof assistant. By encoding the semantics of LLVM IR and the transformations performed by peephole optimizations, we aim to create a framework for verifying the correctness of these optimizations. Specifically, we will focus on a subset of LLVM IR which includes integer arithmetic and memory operations, and formalize a selection of common peephole optimizations. Control flow is out of scope for this project, as we will focus on local tranformations within basic blocks. This work will contribute to the reliability of compiler optimizations and provide insights into the design of future optimization passes.

Additional Key Words and Phrases: Formal Methods, Compiler Optimization, LLVM, Lean4

1 Background

Peephole optimization is a key class of compiler optimizations that targets small instruction sequences, typically within a basic block, to improve performance and reduce code size. These optimizations are usually implemented as pattern-matching rules that identify specific instruction sequences and replace them with more efficient alternatives. Ensuring the correctness of these transformations is critical, as any incorrect optimization can cause erroneous program behavior.

In LLVM, a widely used compiler infrastructure, peephole optimizations are mainly implemented in the InstCombine pass. As of this writing, InstCombine contains over 50,000 lines of code—double its size from ten years ago. Its growing complexity makes maintenance and correctness verification increasingly difficult. Yang *et al.* found InstCombine to be among the most bug-prone components in LLVM [2]. This underscores the need for more rigorous correctness guarantees for peephole optimizations in LLVM.

Alive2 is a formal verification framework based on SMT solvers, specifically designed to verify LLVM optimizations. However, Alive2 has several limitations, including timeouts for complex optimizations and lack of support for multi-width bit vectors, which are common in LLVM optimizations.

2 Proposed Formalization

In this proposal, we aim to formalize a subset of LLVM IR and a selection of peephole optimizations in Lean4 [1], a proof assistant. Table 1 lists the scope of the proposed formalization.

2.1 Instructions

We focus on a subset of LLVM IR with integer arithmetic and memory operations that are typical targets for peephole transforms. This subset includes `add`, `sub`, `mul`, `div`, `select`, `load`, and `store`, together with relevant flags (*e.g.* `nuw`, `nsw`) that affect semantics.

Author's Contact Information: Hongxu Xu, Cheriton School of Computer Science, University of Waterloo, Waterloo, Canada, hongxu.xu@uwaterloo.ca.

This work is licensed under a Creative Commons Attribution 4.0 International License.

Table 1. Scope of the Proposed Formalization in Lean4

Category	Target functionalities and concepts
Instructions	`add`, `sub`, `mul`, `div`, `select`, `load`, `store`; flags: `nsw`, `nuw`
Data types	Arbitrary-width integers (*e.g.* `i32`, `i64`), pointer types; special values: `poison`, `undef`
Memory model	Basic locations/addresses; operational semantics of `load`/`store`; no aliasing or complex hierarchies
Optimizations	Arithmetic identities (e.g. `add x,0` → `x`); constant folding (e.g. `select true,x,y` → `x`)
Properties	Refinement: $P_{opt} \sqsubseteq P_{orig}$

2.2 Data Types

We formalize integer types of varying widths (*e.g.*, `i32`, `i64`) and pointer types, and model `poison` and `undef` to capture LLVM's semantics relevant to optimization correctness.

2.3 Memory Model

The memory model captures basic locations, addresses, and the operational effects of `load` and `store`. We omit aliasing and complex hierarchies in this initial formalization.

2.4 Peephole Optimizations

We formalize common peephole optimizations that target arithmetic and memory patterns, e.g. `add x,0` → `x` and constant folding of `select` when the condition is constant.

2.5 Properties to Verify

For each optimization we prove a refinement property: the optimized program refines the original ($P_{opt} \sqsubseteq P_{orig}$), i.e. it is semantically equivalent or more defined where the original may yield `poison` or `undef`.

3 Roadmap

We plan to complete this project in four phases over eight weeks.

Phase 1: Foundations (Week 1–2). Goal: A working interpreter.

- Define Inductive Instruction and State.
- Implement eval : `List Instruction → State → Option State`.
- Write unit tests in Lean to confirm eval works for simple programs.

Phase 2: The Optimizer (Week 3–4). Goal: A working transformation function.

- Implement optimize function focusing on one pattern first: (X + C1) + C2.
- Verify experimentally that optimize produces cleaner code on test inputs.

Phase 3: Verification (Week 5–7). Goal: The Green Checkmark.

- Prove simple lemmas about BitVec addition (e.g., associativity).
- Define the main theorem: theorem soundness : $\forall p\ s,$ `eval (optimize` p`)` s = `eval` $p\ s$.
- Prove the theorem for the single implemented pattern.
- Stretch goal: Add nsw flag support and prove optimizations do not drop flags incorrectly.

Phase 4: Final Polish (Week 8). Goal: Submission.

- Clean up code and add comments explaining the design rationale.
- Final tests, documentation, and prepare submission materials.

References

[1] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28: 28th International Conference on Automated Deduction, Virtual Event, July 12–15, 2021, Proceedings*. Springer-Verlag, Berlin, Heidelberg, 625–635. doi:10.1007/978-3-030-79876-5_37

[2] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN conference on Programming language design and implementation*. 283–294.